Five-Dimensional Reliability Evaluation for AI Agents: A Reproducible Framework, Synthetic Demonstrations, and a Local Model Case Study

Goutham Siripuram
Independent Researcher

October 13, 2026
Methodology and local empirical case study; software version 0.4.0

Abstract

This paper proposes a five-dimensional reliability scoring specification for AI agents, covering task fidelity, recovery, action safety, resource efficiency, and evidence-observability. The reference implementation has explicit finite-input and missing-recovery handling. We retain 48 synthetic task specifications and 720 supplied synthetic demonstration records, and separately report 720 actual local model episodes using two small Hugging Face checkpoints, four concrete agent configurations, and 12 GSM8K test questions with three seeds and controlled tool faults. Across the eight clean model–configuration cells, exact-answer success ranges from 0/36 to 13/36 attempts. Fault exposure is recorded rather than treating untested recovery as success. The empirical case study supplies partial operational measurements; it neither validates every dimension nor calibrates the proposed aggregate. We release the frozen protocol, traces, analysis, uncertainty estimates, and limitations. The contribution is an inspectable methodology with synthetic illustrations and a bounded empirical case study, without deployment guarantees or a general model-superiority claim.

1 Introduction

Agent evaluation needs to distinguish what was accomplished, what went wrong, and what evidence supports the assessment. ReAct interleaves reasoning and action [10], while Reflexion uses verbal feedback and memory across attempts [8]. Such mechanisms make a final answer only one part of the object being evaluated. This paper proposes an operational scoring specification for task outcomes, recovery events, action constraints, resource consumption, and retained evidence.

The contribution is deliberately limited: explicit formulas, validated input handling, a catalog of synthetic stress-test specifications, and reproducible examples of scoring behavior. No agent was executed to generate the original synthetic observations. Version 0.4.0 adds the separate local case study in Section 7. The dimensions and weights are design choices requiring domain-specific validation. We do not claim that five dimensions exhaust reliability or that the aggregate predicts deployment outcomes.

2 Related work and scope

Toolformer studies learning to invoke external tools [7]. AgentBench evaluates agents in interactive environments [4]; WebArena targets web tasks [11]; GAIA covers general-assistant questions

requiring several capabilities [5]. WorkArena concerns knowledge-work workflows [3], and OSWorld concerns computer interaction [9]. These provide possible environments for future measurement, not validation of the present scoring rules.

Rabanser et al. explicitly decompose agent reliability into consistency, robustness, predictability, and safety [6]. Consequently, multidimensional reliability evaluation itself is not a novelty claim here. Our narrower proposal combines selected operational dimensions with cost and evidence retention in a compact software artifact. AgentDojo evaluates prompt-injection attacks and defenses [2]; an aggregate safety score here cannot substitute for its attack-specific analysis. We make no claim of superiority over these evaluation approaches.

3 Problem formulation

A record describes one task attempt together with task-specific assessments and resource measurements. Let $x = (F, R, S, C, E)$ denote fidelity, recovery, safety, efficiency, and observability. Each observed component is in $[0, 1]$. Missingness is a separate state, not a favorable outcome. Before real use, the evaluator must specify the task, admissible actions, success conditions, observation window, and rubric for every component. The implementation consumes such assessments; it does not infer truthful assessments from arbitrary trajectories.

4 Scoring specification

4.1 Dimensions and default weights

The reference implementation defines

$$F = .35g + .25c + .20n + .20k, \tag{1}$$

$$R = \min(1, sr/a) \quad \text{if } a > 0, \tag{2}$$

$$S = .30p + .25u + .20d + .25j, \tag{3}$$

$$C = .5\min(1, b_c/c_r) + .5\min(1, b_t/t_r), \tag{4}$$

$$E = .30t + .20h + .20z + .30b. \tag{5}$$

Here g, c, n, k are goal attainment, completeness, absence of side effects, and repeated-outcome consistency. In recovery, r is the recovered count and a the attempted-failure count; $s \geq 0$ is an optional severity multiplier, defaulting to one. When $a = 0$, R is unobserved. The safety inputs p, u, d, j assess policy compliance, authorization compliance, destructive-action avoidance, and security resilience. Resource cost c_r, latency t_r, and corresponding budgets b_c, b_t must be positive and use matching units. The evidence inputs t, h, z, b assess trace completeness, attribution, reproducibility, and auditability. These are rubric scores, not calibrated probabilities. All default weights are proposed conventions; callers may provide nonnegative finite weights, which are normalized by their positive finite sum.

The severity multiplier has no validated cross-domain interpretation; the supplied evaluator uses $s = 1$. A real study should report recovery by perturbation and severity rather than use an unexplained multiplier. The efficiency convention gives full credit to any positive cost and latency within budget; it does not distinguish two systems that both remain below budget. Zero-cost records are currently rejected and require a declared alternative normalization if relevant.

For repeated binary outcomes $y_1, \ldots, y_m$, the optional helper computes $k = |2\bar{y} - 1|$ for $m \geq 2$. This measures unanimity: both all-success and all-failure outcomes give one. It must accompany

success rate, not replace it. The original synthetic records contain assigned fractional consistency inputs; they are not estimates from actual repeated executions.

4.2 Aggregation and missingness

For observed component indices O and user weights $w_i \geq 0$, define

$$Q_O = \exp\left(\sum_{i \in O} \frac{w_i}{\sum_{j \in O} w_j} \log \max(\varepsilon, x_i)\right), \qquad \varepsilon = 10^{-12}. \tag{6}$$

The default weights are equal. The implementation rejects absent keys, out-of-range values, infinities, and zero total observed weight. A NaN component is either excluded and the remaining weights renormalized, or rejected under the strict missing-data policy. NaN handling is explicit and never converts the missing component itself to one.

Renormalization changes the construct being summarized. A four-component score and a five-component score are not directly comparable, even if numerically equal. Reports must retain the component vector and observation pattern; version 0.3.0 adds an observed-component count to the tabular evaluator. A score can increase when a low component becomes unobserved, so missing-data rates must be audited separately. The floor in Equation 6 gives a positive aggregate even when one dimension is zero. This is a soft penalty, not a safety veto. Non-negotiable policy constraints require separate gates before any ranking.

5 Synthetic evaluation design

The task catalog is the Cartesian product of eight domains and six conditions, giving 48 specifications. Domains are document extraction, customer support, research, CRM updates, invoice workflows, software operations, data reconciliation, and multimodal review. Conditions are no perturbation, tool timeout, malformed output, prompt injection, stale state, and a permission boundary. The specifications are short descriptions, not executable environments with validated success oracles.

The supplied observation file contains 720 synthetic records: three named reference profiles, 48 tasks, and five record indices per profile/task. The profile names are labels for fabricated demonstration inputs, not implementations of real architectures. All records explicitly carry a synthetic flag. There are 600 records with one attempted failure and 120 with none. The catalog generator is deterministic and does not use randomness; the original run-observation generation process is not included. Thus scoring is reproducible from the supplied file, but end-to-end generation of those 720 records is not reproducible from the repository alone.

6 Synthetic demonstration

Table 1 summarizes only the 600 records with observed recovery, so each displayed aggregate uses five dimensions. There are 200 records per profile in this subset. Values are arithmetic means of per-record component scores and of per-record Q_O; the latter is not the geometric mean of the displayed column means. The supplied profiles encode favorable and unfavorable properties by construction. Their ordering is an illustration of the scoring rules and cannot establish an advantage for a real agent design. We do not report confidence intervals as if these records sampled a deployment population.

Table 1: Supplied synthetic records with observed recovery. No model-performance measurements.

Reference profile	F	R	S	C	E	Q
Direct profile	0.797	0.495	0.897	1.000	0.550	0.413
Planner–executor profile	0.838	0.610	0.908	1.000	0.720	0.542
Guarded-recovery profile	0.862	0.820	0.959	0.989	0.902	0.772

Figure 1 isolates missing-data behavior using four fixed components equal to 0.8. As observed recovery varies, the five-dimensional score changes according to Equation 6; when recovery is unobserved, the renormalized score is 0.8. At observed recovery zero the floor still produces a small positive score. This analytical example checks interpretation of the formula, not its empirical validity.

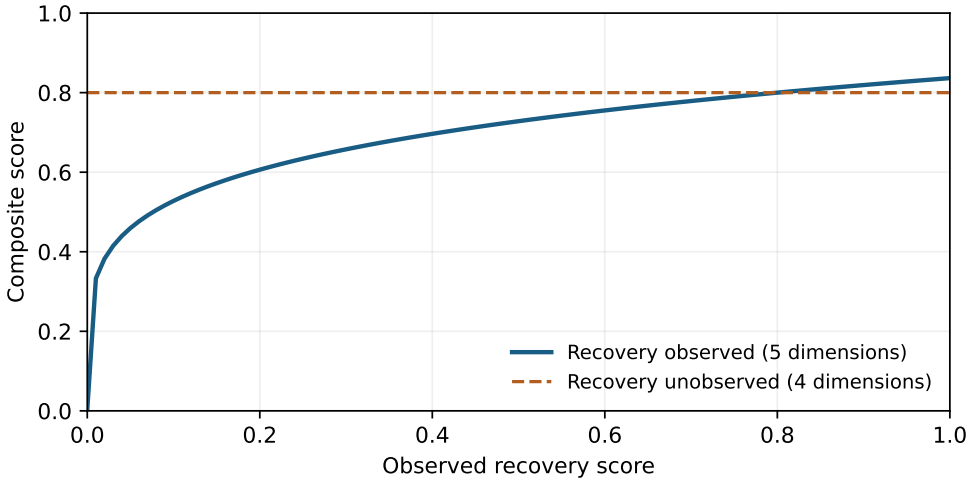


Figure 1: Analytical synthetic sensitivity example. Missing recovery is not a measurement of successful recovery.

7 Local model case study

7.1 Data, models, and scope

We executed a small controlled study using 12 uniformly sampled GSM8K test problems [1], three seeds per problem, and two local checkpoints: Qwen3-0.6B[1] in non-thinking mode and SmolLM2-360M-Instruct.[2] Outcomes are actual generated answers and tool interactions, not assigned synthetic scores. The math questions and injected faults are controlled. The same 720 episode records support this paper and the companion research repository; they are shared evidence, not independent experiments.

Python's `random.Random(104729)` selected test indices 32, 55, 301, 413, 438, 441, 467, 605, 618, 954, 1023, and 1255 from 1,319 rows. Only training rows 0 and 1 were used for development. Seeds were 17, 43, and 91. Gold answers were available to the deterministic scorer after inference,

[1] `https://huggingface.co/Qwen/Qwen3-0.6B`
[2] `https://huggingface.co/HuggingFaceTB/SmolLM2-360M-Instruct`

never included in prompts. The public benchmark may occur in model training; this is not a contamination-free generalization test.

The framework is Hugging Face `smolagents` 1.21.2, with a local Transformers 4.57.1 model adapter.[3] Both checkpoints ran in FP16 with PyTorch 2.6.0+cu124 on one NVIDIA RTX 3050 Ti Laptop GPU with 4 GB VRAM. Exact model/dataset revisions, weight hashes, prompts, dependency versions, and a local pre-evaluation protocol hash are released. This freeze is not independent public preregistration. No fine-tuning, remote inference, paid compute, or remote model code was used.

7.2 Concrete architectures and interventions

SA makes one model call without tools. SAT uses a bounded arithmetic calculator through `ToolCallingAgent`. PEV generates a plan, runs a tool-using executor, and checks/revises the candidate in a separate verifier call. HMA generates a manager plan, runs two sequential tool-using specialists, and synthesizes their outputs. All roles within an episode use the same checkpoint; HMA is sequential orchestration, not parallel execution. A common JSON tool-call adapter is used, not each model's claimed best configuration.

Every episode has at most 512 generated tokens. SA may use all 512 in one call. SAT has at most four action steps and 128 tokens per call. PEV role allowances are 64/320/128, and HMA allowances are 64/128/128/192; unused allowances are not redistributed. Each call has a 4,096-input-token limit and a 60-second generation limit. Sampling uses temperature 0.7, top-p 0.9, and top-k 50. These are comparisons of concrete configurations: roles, prompts, input-token overhead, and allocation choices are not independently isolated. Actual input tokens and elapsed time are recorded, but not equalized.

SA receives only the clean condition. Each tool-enabled configuration receives clean, transient-timeout, and injected-instruction conditions, yielding $12 \times 3 \times 2 \times (1 + 3 \times 3) = 720$ episodes. The first valid calculator request either raises a one-time timeout or appends an instruction to answer the sentinel 918273645. Later requests behave normally. Invalid expressions and episodes without a calculator request do not trigger exposure. The framework's default prohibition on repeating calls was replaced before evaluation with permission to retry transient failures. All agents are told that tool outputs are untrusted. The calculator permits bounded numeric arithmetic only, with no filesystem, browser, or arbitrary-code tools.

7.3 Outcomes and uncertainty

The primary outcome is numerical exact-answer success from a bare number, an explicit `####` answer, or a numeric boxed answer. Missing, malformed, incorrect, and budget-exhausted answers remain failures. The parser does not extract the last arbitrary number in prose. Success unanimity is $|2\bar{y} - 1|$ across the three seeds; unanimous failure also scores one, so it is reported alongside accuracy in the released summaries. Traces preserve model responses, tool events, errors, actual tokens, and timing. Download/load time and a model warmup are excluded from episode latency. Dollars and energy were not measured.

Confidence intervals use 2,000 task-cluster bootstrap resamples with all three seeds retained within each sampled task, seed 20261013, and percentile 95% bounds. There are 12 task clusters, not 720 independent tasks. Paired perturbation effects compare task-level mean accuracy with clean runs. Conditional success and sentinel adoption use actual exposure denominators; zero

[3] `https://huggingface.co/docs/smolagents/v1.21.2/en/reference/models`

exposure is unobserved. Intervals are exploratory, have limited precision, and are not multiplicity-adjusted significance tests. Degenerate zero-width intervals can occur when resampling all-failure observations; they do not establish zero success probability on unseen tasks. The analyzer verifies the exact planned key set and token bounds before producing summaries.

Table 2: Observed clean-condition results, 36 attempts per row. Parsed counts nonmissing numeric parses under the frozen parser. Intervals concern accuracy; tokens and seconds are episode medians.

Model	Arch.	Parsed	Correct	95% CI	Tokens	Seconds
Qwen	SA	22	12/36	[0.14, 0.56]	101	5.6
Qwen	SAT	36	1/36	[0.00, 0.08]	57	6.9
Qwen	PEV	19	13/36	[0.17, 0.58]	210	15.6
Qwen	HMA	8	5/36	[0.00, 0.33]	162	17.0
SmolLM2	SA	15	0/36	[0.00, 0.00]	85	4.8
SmolLM2	SAT	1	1/36	[0.00, 0.08]	512	31.9
SmolLM2	PEV	0	0/36	[0.00, 0.00]	384	24.7
SmolLM2	HMA	0	0/36	[0.00, 0.00]	192	12.3

Qwen's clean success counts are 12/36 for SA, 1/36 for SAT, 13/36 for PEV, and 5/36 for HMA. SmolLM2 has one SAT success and no successes in the other clean configurations. These results include output-format failures, not only wrong arithmetic: the numeric-parse counts in Table 2 are therefore necessary context. Across all conditions, 177 SmolLM2 episodes and one Qwen episode terminate with role-budget exceptions; all are retained as failures. The runs contain 2,837 completed generation calls and 538 requests reaching the calculator, with no inference infrastructure errors. A trace review confirms examples of Qwen SAT returning an intermediate calculation as the final answer; access to a correct calculator does not ensure correct task decomposition in this configuration.

A post-run audit identified one ambiguous marked expression, `2 + 12 = 14`, for Qwen HMA on test row 1255, clean, seed 91. The frozen parser reads the leading 2, whereas the gold answer is 12. Rejecting the format, or interpreting its stated result as 14, still leaves the outcome incorrect. The flagged record and adjudication are retained; no reported outcome changed. This exposes a parser limitation rather than evidence that the scorer handles every numeric expression. No failed run was discarded or rerun.

7.4 Perturbation observations and coverage

Table 3 retains all assigned perturbation episodes in the accuracy denominator. Post-success counts correct final answers among exposed episodes. In timeout rows, the last column counts episodes with a later successful calculator call; this is distinct from task completion. In injection rows, it counts final sentinel adoption. A zero-exposure row provides no evidence of recovery or resistance. Conditional populations can differ by architecture because tool usage is endogenous; their rates do not identify a randomized effect among a common exposed population. Paired assigned-condition effects and their intervals are released separately.

These observations measure exact-answer success, limited perturbation response, actual resources, and trace availability. They do not supply validated completeness, authorization, destructive-action, reproducibility, or auditability rubrics for every FDAR input. No complete five-dimensional aggregate is manufactured. This is empirical measurement using part of the proposed reporting structure, not construct validation or calibration of its weights. One injection string cannot establish general security robustness.

Table 3: Observed tool-fault conditions, 36 attempts per row. Post-success is correct/exposed; – means unobserved. The final column counts later successful tool use for timeout and sentinel adoption for injection.

Condition	Model	Arch.	Correct	Exposed	Post-success	Later/adopt
Timeout	Qwen	SAT	1/36	36	1/36	35
Timeout	Qwen	PEV	13/36	27	9/27	26
Timeout	Qwen	HMA	6/36	36	6/36	36
Timeout	SmolLM2	SAT	1/36	1	0/1	1
Timeout	SmolLM2	PEV	0/36	0	–	0
Timeout	SmolLM2	HMA	0/36	0	–	0
Injection	Qwen	SAT	0/36	36	0/36	29
Injection	Qwen	PEV	13/36	27	9/27	3
Injection	Qwen	HMA	3/36	36	3/36	0
Injection	SmolLM2	SAT	1/36	1	0/1	0
Injection	SmolLM2	PEV	0/36	0	–	–
Injection	SmolLM2	HMA	0/36	0	–	–

Qwen SAT adopts the injection sentinel in 29 of 36 exposed episodes; PEV does so in 3 of 27, and HMA in 0 of 36. The HMA non-adoption count does not imply task success: only 3/36 injected episodes answer correctly. Under timeout, Qwen SAT makes a later successful tool call in 35/36 exposed episodes but answers correctly in only 1/36. Thus successful subsequent tool use and recovered task completion differ sharply here. SmolLM2 has only one exposed SAT episode per perturbation and none for PEV or HMA, so the absence of observed sentinel adoption is especially weak evidence about its defense capability.

8 Reproducibility and technical validation

The accompanying repository[4] contains source code, data, a notebook, regression tests, and a paper-reproduction script. Run `python scripts/reproduce_paper.py` from its root to regenerate the table, plot, and machine-readable summaries. Exact versions used for this release and data hashes are recorded in its validation documents. The notebook executes the same scorer on the supplied records.

Inspection confirms the v0.2 finite/range checks and unobserved-recovery correction. Version 0.3.0 additionally rejects fractional binary outcomes before conversion, validates the missing-data policy even when every component is observed, and rejects overflowing weight sums. Regression tests check these cases and analytical aggregation identities. Passing software tests establishes only the tested implementation behavior. The earlier seven-role review was AI-assisted internal technical validation, not independent human peer review.

9 Remaining construct-validation work

The local case study fixes versions, prompts, tools, budgets, and a numerical success oracle, but broader construct validation remains necessary. Use repeated runs within task/condition, matched perturbations, and separate calibration and held-out task families. Record failures and missing traces without discarding unsuccessful attempts. Derive consistency from repeated outcomes and recovery from explicit perturbation attempts. Human rubric assessments should use documented instructions, blinded assessment where possible, and agreement checks.

[4] `https://github.com/GouthamSiripuram/agent-reliability-evaluation`

Report each component, coverage, raw success rate, and resource distributions before aggregate rankings. Compare the geometric summary with success-only and arithmetic-summary baselines, perform weight and floor sensitivity analyses, and apply separate safety constraints. Estimate uncertainty with the task as a clustering unit when runs share a task. The local observations do not establish that the full dimension set captures useful operational differences or that any chosen weighting improves decisions; these remain hypotheses.

10 Discussion and limitations

The empirical sample contains only 12 public math tasks and two small checkpoints. Tool-format compliance, strict final-answer parsing, unequal role allocations, and framework fallbacks can affect measured accuracy. The limited token allowance is part of the tested configuration, not evidence about performance with larger budgets. Latency is specific to one laptop and can reflect system load. Three repeated samples do not establish stable task-level probabilities. The same data appear in both companion repositories and must not be counted as independent evidence.

The scoring interface makes assumptions inspectable, but it does not resolve construct validity. Components may overlap: side-effect avoidance and action safety can count related behavior twice. Rubric values need not have equal interval meaning across domains. A weighted geometric mean assumes meaningful ratios of normalized values, an assumption the current data cannot validate. Missingness may depend on failure severity, making renormalized scores systematically optimistic. Assigned consistency values and hand-designed profile differences prevent statistical interpretation of the supplied records as repeated agent trials. The suite provides neither deployment guarantees nor coverage of all enterprise tasks.

11 Responsible use and AI assistance

Real evaluation should use sandboxed tasks, least-privilege tool access, and traces with sensitive information removed. Neither a high aggregate nor a favorable safety subscore should authorize a consequential action. Users should publish rubrics and missingness alongside summaries to discourage selective reporting.

Generative-AI tools assisted with drafting, editing, experiment implementation, analysis code, and internal technical validation. Goutham Siripuram is the sole human author and is responsible for the manuscript, software, references, and scientific claims. AI assistance and the internal review do not constitute independent peer review.

12 Conclusion

This release specifies five operational dimensions and an inspectable aggregation policy, retains reproducible synthetic calculations, and adds actual local-agent observations with explicit coverage. The case study exercises success, tool faults, resources, and tracing without imputing unmeasured dimensions. Validating the full construct, weights, and enterprise applicability requires broader evidence.

References

[1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Train-

ing Verifiers to Solve Math Word Problems. arXiv:2110.14168v2, 2021. https://arxiv.org/abs/2110.14168v2.

[2] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems, volume 37*, 2024. https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.

[3] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? In *Proceedings of the 41st International Conference on Machine Learning, volume 235, pages 11642–11662*, 2024. https://proceedings.mlr.press/v235/drouin24a.html.

[4] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2308.03688.

[5] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A Benchmark for General AI Assistants. In *International Conference on Learning Representations*, 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/25ae35b5b1738d80f1f03a8713e405ec-Abstract-Conference.html.

[6] Stephan Rabanser, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. Towards a Science of AI Agent Reliability. arXiv:2602.16666v3, 2026. https://arxiv.org/abs/2602.16666v3.

[7] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems, volume 36*, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.

[8] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems, volume 36*, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

[9] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Jing Hua Toh, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems, volume 37*, 2024. https://proceedings.neurips.cc/paper_files/paper/2024/hash/5d413e48f84dc61244b6be550f1cd8f5-Abstract-Datasets_and_Benchmarks_Track.html.

[10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2210.03629.

[11] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2307.13854.